UNIT-2

Divide and Conquer Method: General Method – binary search –maximum and minimum – merge sort - quick sort – Strassen's Matrix multiplication**. Greedy Method:** General method – knapsack problem – minimum spanning tree algorithms – single source shortest path algorithm – scheduling, optimal storage on tapes, optimal merge patterns

1. Explain about Divide and conquer method with algorithm.

- ✓ Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs into 'k' distinct subsets, 1<k<=n, yielding 'k' sub problems. These sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole.
- ✓ If the sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied.
- ✓ Often the sub problems resulting from a divide-and-conquer design are of the same type as the original problem.
- ✓ For those cases the re application of the divide-and-conquer principle is naturally expressed by a recursive algorithm.
- ✓ D And C(Algorithm) is initially invoked as D and C(P), where 'p' is the problem to be solved.
- ✓ Small(P) is a Boolean-valued function that determines whether the i/p size is small enough that the answer can be computed without splitting.
- ✓ If this so, the function 'S' is invoked.
- ✓ Otherwise the problem P is divided into smaller sub problems.
- ✓ These sub problems P1,P2,……Pk are solved by recursive application of DAndC.
- ✓ Combine is a function that determines the solution to p using the solutions to the 'k' sub problems.
- ✓ If the size of 'p' is n and the sizes of the 'k' sub problems are n1,n2,…….nk, respectively, then the computing time of D And C is described by the recurrence relation.

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1)+T(n_2)+\ldots\ldots\ldots\ldots+T(n_k)+f(n); & \text{otherwise.} \end{cases}$$

Where T(n) ❼ Time for D And C on any i/p of size 'n'.

g(n) ❼ Time of compute the answer directly for small i/ps.

f(n) ❼ Time for dividing P & combining the solution to sub problems.

Pseudo Code Algorithm for D&C Method:

```
Algorithm D And C(P)
{
if small(P) then return S(P);
else
  {
    divide P into smaller instances
        P1,P2,…….,Pk, k>=1;

    Apply D And C to each of these sub problems;
    return combine (D And C(P1), D And C(P2),…….,D And C(Pk));
  }
```

```
}
```

2. Explain about Binary Search using Divide and Conquer method.

Divide and conquer method:

- ✓ Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs into 'k' distinct subsets, 1<k<=n, yielding 'k' sub problems. These sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole.
- ✓ If the sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied.

Problem statement:

Let a_i, 1<=i<=n be a list of elements that are sorted in non decreasing order. Consider the problem of determining whether a given element x is present in the list. If x is present we are to determine a value j such that aj=x. if x is not in the list then j is to be set zero.

Algorithm: Iterative Binary search

```
Algorithm Binsearch(a,n,x)
// Given an array a[1:n] of elements
 in non-decreasing
//order, n>=0,determine whether 'x' is
present and
// if so, return 'j' such that x=a[j]; else return 0.
{
low:=1; high:=n;
while (low<=high) do
{
        mid:=(low+high)/2;
        if (x<a[mid]) then
            high:=mid-1;
        else if(x>a[mid]) then
             low=mid+1;
    else
            return mid;
}
   return 0;
}
```

Algorithm Recursive Binary search:

```
Algorithm BinSrch(a,i,l,x)
// Given an array a[i:l] of elements in
non-decreasing
//order, n>=0,determine whether 'x' is
present and
// if so, return 'j' such that x=a[j]; else
return 0.
{
      if(l=i) then // Small (p)
       {
             if(x=a[i]) then return i;
       else return 0; 10
       }
else
  {        //  Reduce    P into smaller sub
problem.
           mid: =[ (i+l)/2];
           if (x=a[mid]) then return mid;
         else if (x<a[mid]) then
                return BinSrch(a,i,mid-1,x);
           else                          return
BinSrch(a,mid+1,l,x);
   }
}
```

EXAMPLE :1 Consider the list of elements 2 , 4 , 6, 8,10 are placed in the array Search for the element X=2 and using binary Search.

STEP 1: **Initialize all the elements into array**

A[1]	A[2]	A[3]	A[4]	A[5]
2	4	6	8	10

Where Low= A[1] HIGH= A[5]

STEP 2: **Now find the Mid value using the formula MID = [(low+high)/2]**

Where low =1 and high =5

MID= [(1+ 5)/2]

MID=[(6)/2] => MID = 3 therefore A[3] =6.

A[1]	A[2]	A[3]	A[4]	A[5]
2	4	6	8	10
low		mid		high

STEP 3: **Now check for the condition X= a[mid] where we are going to search for the element X=2.**

A[1]	A[2]	A[3]	A[4]	A[5]
2	4	6	8	10
low		mid		high

search element X=2 check for X= a[mid] => 2=a[3] => 2= 6 case false
There search element is not the mid element.

STEP 4: **Now check for the condition X< a[mid] where we are going to search for the element X=2.**

A[1]	A[2]	A[3]	A[4]	A[5]
2	4	6	8	10
low		mid		high

search element X=2 check for X< a[mid] => 2<a[3] => 2< 6 condition TRUE.

If the condition is true the change high:=mid-1; high= 3-1 => high=a[2]=>4

The new updated high is 4

A[1]	A[2]	A[3]	A[4]	A[5]
2	4	~~6~~	~~8~~	~~10~~
low		high		

STEP 5: **Now again find the Mid value using the formula MID = [(low+high)/2]**

Where low=1 high= 2 MID= [(1+ 2)/2] MID=[(3)/2] => MID = 1 therefore A[1] =2

A[1]	A[2]	A[3]	A[4]	A[5]
2	**4**	~~6~~	~~8~~	~~10~~
Low	mid	high		

STEP 6: **Now check for the condition X= a[mid] where we are going to search for the element X=2.**

A[1]	A[2]	A[3]	A[4]	A[5]
2	**4**	~~6~~	~~8~~	~~10~~
Low	mid	high		

search element X=2 check for X= a[mid] => 2=a[1] => 2= 2 case TRUE
There search element is found as the mid element.

ELEMENT FOUND

Example:2 Let us select the 14 entries. [-15,-6,0,7,9,23,54,82,101,112,125,131,142,151]

- Place them in a[1:14], and simulate the steps Binsearch goes through as it searches for different values of 'x'.
- Only the variables, low, high & mid need to be traced as we simulate the algorithm.
- We try the following values for x:151,-14 and 9.

for 2 successful searches & 1 unsuccessful search.

Table shows the traces of Bin search on these 3 steps:

X=151	low	high	mid
	1	14	7
	8	14	11
	12	14	13
	14	14	14

Element Found

x=-14	low	high	mid
	1	14	7
	1	6	3
	1	2	1
	2	2	2
	2	1	Not found

x=9	low	high	mid	
	1	14	7	
	1	6	3	
	4	6	5	Element Found

Analysis:

Time Complexity for Successful Search:

Best case :T(n)=O(1)

Average case :T(n)=O(log n)

Worst case :T(n)=O(log n)

Time Complexity for Unsuccessful Search:

T(n)>=O(log n)

3. Explain about Finding Maximum and Minimum Element in the List using Divide and Conquer technique.

Divide and conquer method:

Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs into 'k' distinct subsets, 1<k<=n, yielding 'k' sub problems. These sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole. If the sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied

Problem statement:

A divide and conquer algorithm for this problem would proceed as follows: Let P= (n,a[i],…..a[j]) enote an arbitrary instance of the problem. Here n is the number of elements in the list a[i],….a[j] and we are interested in finding the maximum and minimum of this list.

Let Small(p) be true when n<=2. In this case maximum and minimum are a[i] if n=1. If n=2, the problem can be solved by making one comparison. If the list has more than two elements P has to be divided into smaller instances.

Straight Forward Method:

Algorithm:

```
Algorithm MaxMin(a,n,max,min)
 // set max to the maximum & min to the minimum of a[1:n]
{
        max:=min:=a[1];
        for i:=2 to n do
        {
            if(a[i]>max) then
                max:=a[i];
            if(a[i]<min) then
                min:=a[i];
        }
}
```

Time Complexity:

- Straight forward method requires 2(n-1) element comparison in the best, average & worst cases.

- An immediate improvement is possible by realizing that the comparison a[i]<min is necessary only when a[i]>max is false.
- Hence we can replace the contents of the for loop by,
  - if(a[i]>max) then max:=a[i];
  - else if (a[i]<min) then min:=a[i];
  -

Algorithm: Recursively Finding the Maximum & Minimum

```
Algorithm MaxMin (i,j,max,min)
//a[1:n] is a global array, parameters i & j
//are integers, 1<=i<=j<=n.The effect is to
//set max & min to the largest & smallest value
//in a[i:j], respectively.

 {
        if(i=j) then max:= min:= a[i];
        else if (i=j-1) then // Another case of small(p)
        {
              if (a[i]<a[j]) then
               {
                   max:=a[j];
                    min:=a[i];
                }
            else
               {
                    max:=a[i];
                     min:=a[j];
               }
         }
   else
     {
       // if P is not small, divide P into subproblems.
           // find where to split the set
            mid:=(i+j)/2;
          //solve the subproblems
          MaxMin(i,mid,max.min);
         MaxMin(mid+1,j,max1,min1);
           //combine the solutions
          if (max<max1) then max=max1;
          if(min>min1) then min = min1;
     }
}
```

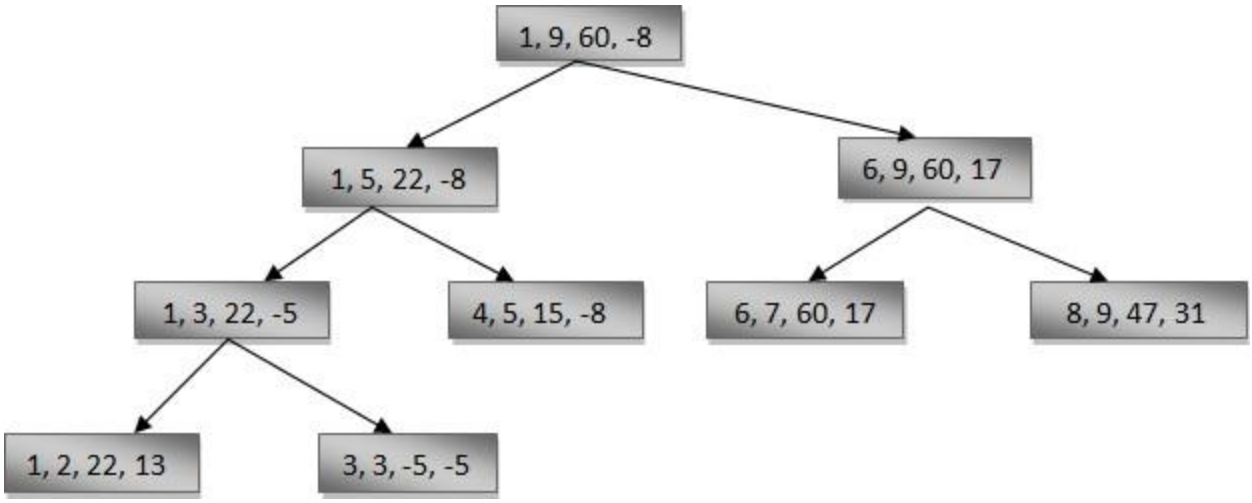
EXAMPLE: 1

The procedure is initially invoked by the statement, MaxMin(1,n,x,y)

- ✓ Suppose we simulate MaxMin on the following 9 elements
- ✓ A: [1] [2] [3] [4] [5] [6] [7] [8] [9]
  - 22 13 -5 -8 15 60 17 31 47

- ✓ A good way of keeping track of recursive calls is to build a tree by adding a node each time a new call is made.
- ✓ For this Algorithm each node has 4 items of information : i,j,max & imin.

- ✓ Examining fig: we see that the root node contains 1 & 9 as the values of i &j corresponding to the initial call to MaxMin.
- ✓ This execution produces 2 new calls to MaxMin, where i&j have the values 1, 5 & 6 ,9 respectively & thus split the set into 2 subsets of approximately the same size.
- ✓ From the tree, we can immediately see the maximum depth of recursion is 4. (including the 1st call)
- ✓ The included numbers in the upper left corner of each node represent the order in which max & min are assigned values.



THE MAXIMUM ELEMENT IS : 60 THE MINIMUM ELEMENT IS : -8

No. of element Comparison:

- If T(n) represents this no., then the resulting recurrence relations is

$$T(n) = \begin{cases} T([n/2]+T[n/2]+2 & n>2 \\ 1 & n=2 \\ 0 & n=1 \end{cases}$$

❼ when 'n' is a power of 2, n=2^k for some +ve integer 'k', then

4. Explain briefly about Merge sort using Divide and Conquer method.

Divide and conquer method:

Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs into 'k' distinct subsets, 1<k<=n, yielding 'k' sub problems. These sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole. If the sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied.

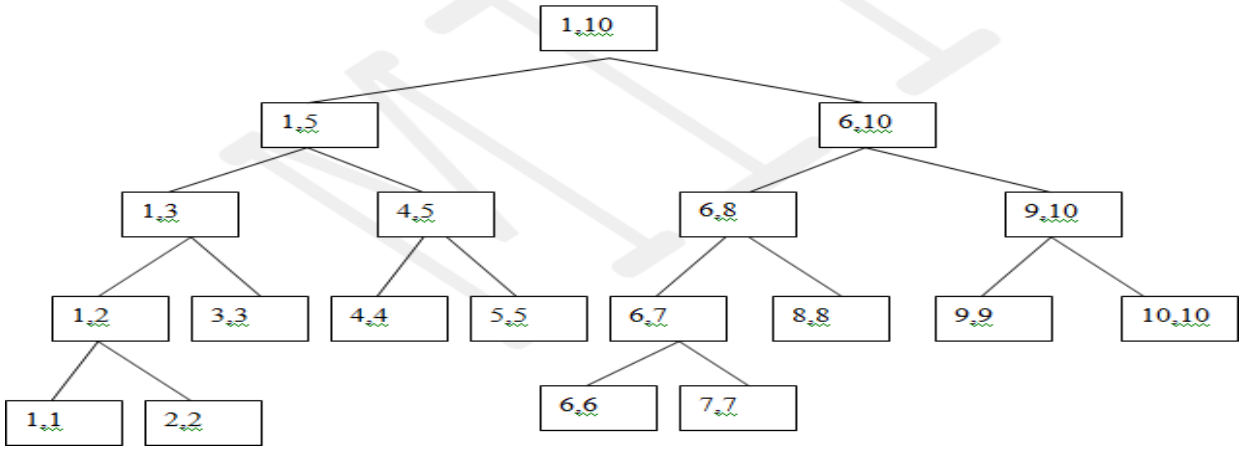
Problem statement: As another example divide-and-conquer, we investigate a sorting algorithm that has the nice property that is the worst case its complexity is O(n log n) This algorithm is called merge sort.

- ✓ We assume throughout that the elements are to be sorted in non-decreasing order.
- ✓ Given a sequence of 'n' elements a[1],…,a[n] the general idea is to imagine then split into 2 sets a[1],…..,a[n/2] and a[[n/2]+1],….a[n].
- ✓ Each set is individually sorted, and the resulting sorted sequences are merged to produce a single sorted sequence of 'n' elements.
- ✓ Thus, we have another ideal example of the divide-and-conquer strategy in which the splitting is into 2 equal-sized sets & the combining operation is the mering of 2 sorted sets into one.

Algorithm For Merge Sort:

```
Algorithm MergeSort(low,high)
//a[low:high] is a global array to be sorted
//Small(P) is true if there is only one element
//to sort. In this case the list is already sorted.
{
        if (low<high) then //if there are more
                                than one element
        {
            //Divide P into subproblems
            //find where to split the set
                mid = (low+high)/2;
            //solve the subproblems.
                    mergesort (low,mid);
                    mergesort(mid+1,high);
            //combine the solutions .
                   merge(low,mid,high);
        }
}
```

Tree for calls of MergeSort:

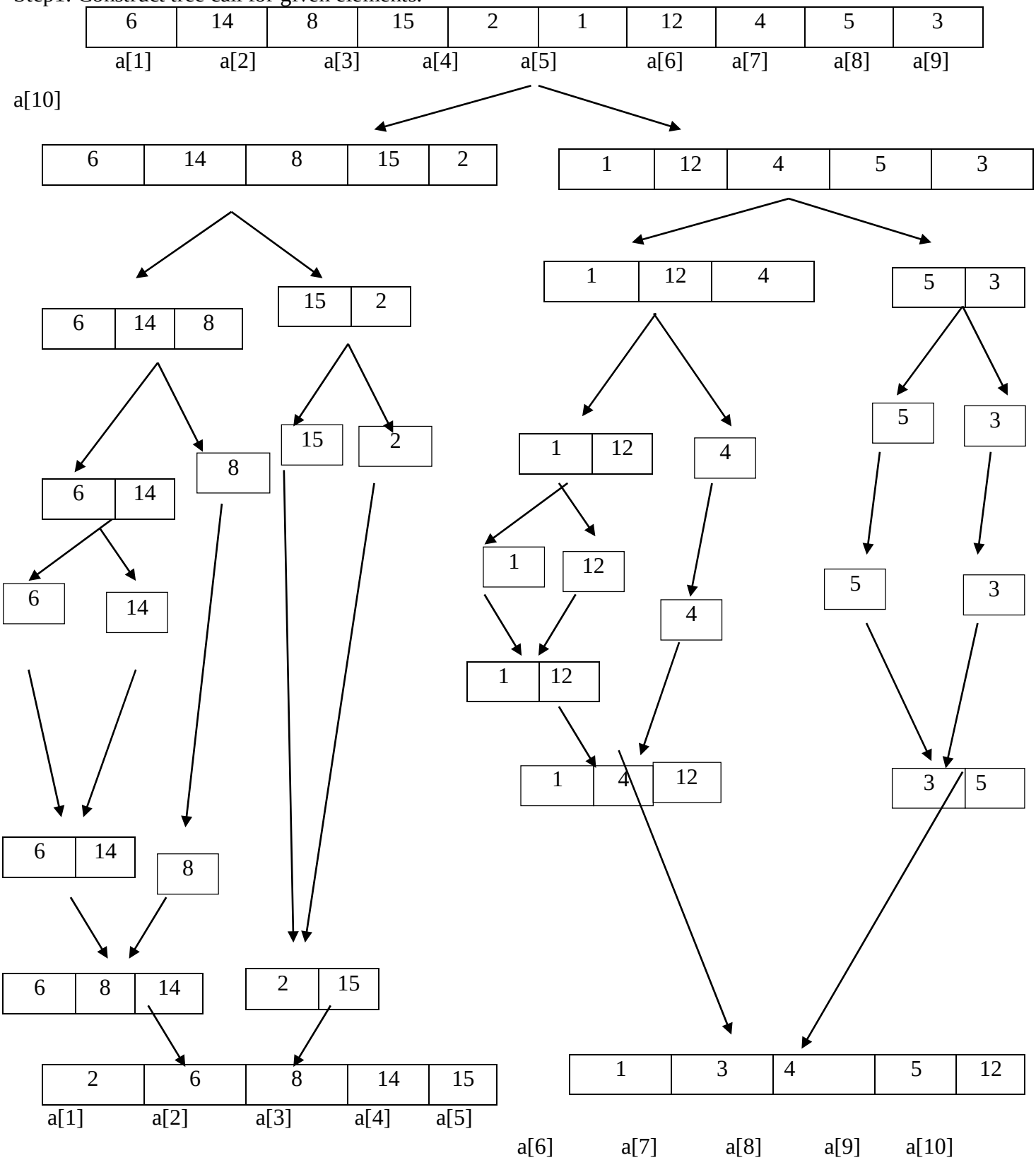


Algorithm: Merging 2 sorted subarrays using auxiliary storage.

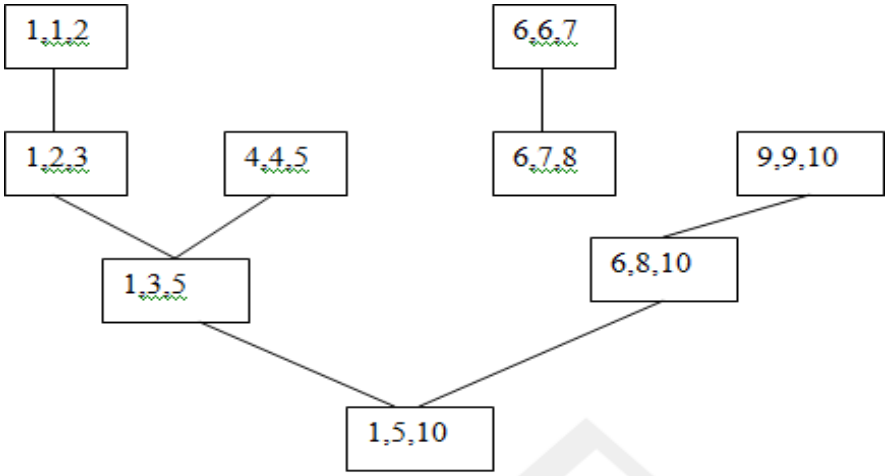
```
Algorithm merge(low,mid,high)
//a[low:high] is a global array containing
//two sorted subsets in a[low:mid]
//and in a[mid+1:high].
The goal is to merge these 2 sets into
//a single set residing in a[low:high].
b[] is an auxiliary global array.
{
    h=high; i=low; j=mid+1;
    while ((h<=mid) and (j<=high)) do
    {
        if (a[h]<=a[j]) then
        {
            b[i]=a[h];
            h = h+1;
        }
        else
        {
            b[i]= a[j];
            j=j+1;
        }
        i=i+1;
    }
if (h>mid) then
    for k=j to high do
    {
        b[i]=a[k];
        i=i+1;
    }
else
    for k=h to mid do
    {
        b[i]=a[k];
        i=i+1;
    }
    for k=low to high do
        a[k] = b[k];
}
```

Example: Consider the list of 10 elements sort them using Merge sort [6,14,8,15,2,1,12,4,5,3]

Step1: Construct tree call for given elements.



The two sorted sub arrays are made. Now merging of two arrays can be made as shown in the figure.



Taking **partition(1,10)**

msort(1,5,10)

2	6	8	14	15
A a[1]	a[2]	a[3]	a[4]	a[5]

i

C **k** c[1]	c[2]	c[3]	c[4]	c[5]

1	3	4	5	12
B b[1]	b[2]	b[3]	b[4]	b[5]

j

Step1 : The function msort(1,5,10) calls the two sorted sub array namely A & B to be merged and stored in the Auxiliary array namely C. Where the A array has Pointer I, B array has a pointer j, Auxiliary array has a pointer k.

Now check for the condition If(a[i]<=b[j]) then c[k]=a[i] and i=i+1 & k=k+1; by case if condition fails c[k]=b[j] and j=j+1 & k=k+1;

Step2: Check for the condition If(a[i]<=b[j]) => if (a[1]<=b[1]) => if(2<=1) condition false

So c[k]=b[j] => c[1]=b[1] (copying the element from B array to C array) increment j=j+1 & k=K+1;

2	6	8	14	15

A a[1] a[2] a[3] a[4] a[5]

i

1				

C c[1] k c[2] c[3] c[4] c[5]

~~1~~	3	4	5	12

B b[1] b[2] b[3] b[4] b[5]

j

Step 3: **: Check for the condition** If(a[i]<=b[j]) => if (a[1]<=b[2]) => if(2<=3) condition true

then c[k]=a[i] => c[2]=a[1] (copying the element from A array to C array) increment i=i+1 & k=K+1;

~~2~~	6	8	14	15

A a[1] a[2] a[3] a[4] a[5]

i

1	2			

C c[1] c[2] k c[3] c[4] c[5]

~~1~~	3	4	5	12

B b[1] b[2] b[3] b[4] b[5]

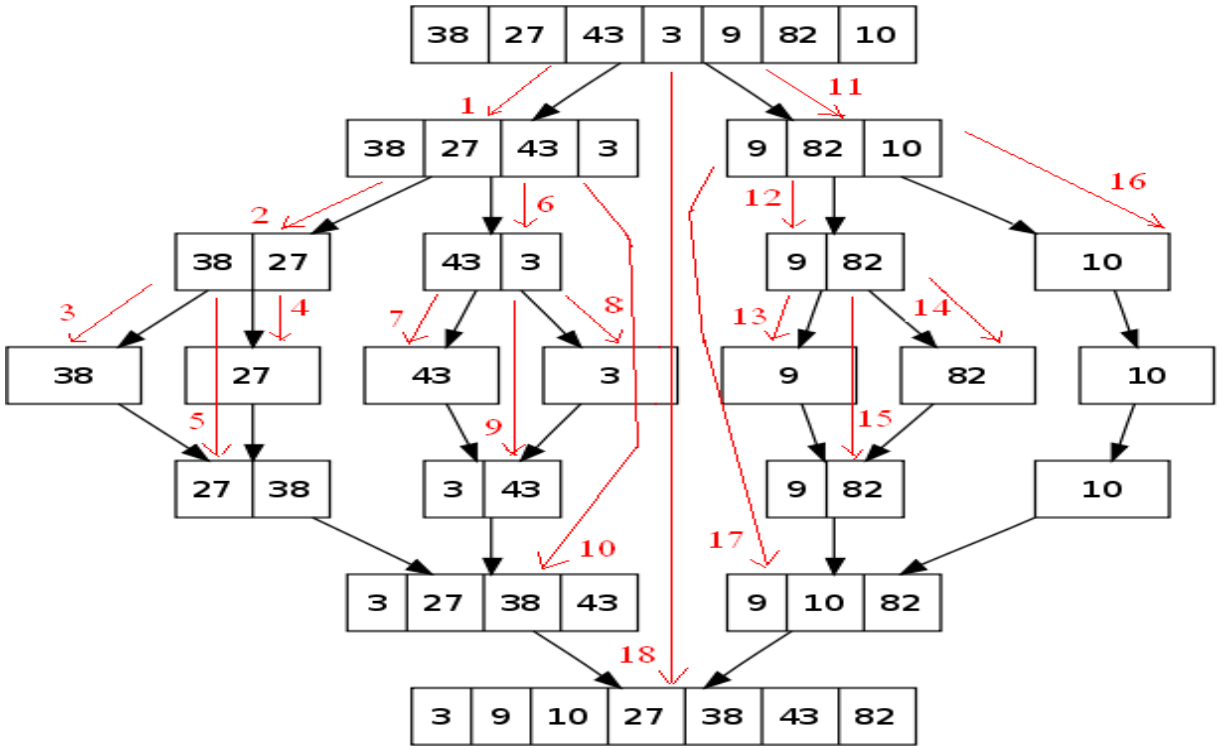
j

Step4: Repeat the steps from step 1 to step 3 to get all the elements in sorted order.

Therefore the sorted elements are

1	2	3	4	5	6	8	12	14	15

EXAMPLE:2 Consider the list of elements 38 27 43 3 9 82 10. Sort them using Merge sort Technique.



TIME COMPLEXITY

- If the time for the merging operations is proportional to 'n', then the computing time for merge sort is described by the recurrence relation.

T(n) = { a n=1,'a' is constant
2T(n/2)+cn n>1,'c' is constant.

It is easy to see that if s^k<n<=2^k+1, then T(n)<=T(2^k+1). Therefore,

T(n)=O(n log n)

ANALYSIS:

BEST CASE: O (nlogn)

WORST CASE: O (nlogn)

AVERAGE CASE: O (nlogn)

5. Explain briefly about Quick sort using Divide and Conquer method.

QUICK SORT:

- The divide-and-conquer approach can be used to arrive at an efficient sorting method different from merge sort.
- In merge sort, the file a[1:n] was divided at its midpoint into sub arrays which were independently sorted & later merged.
- In Quick sort, the division into 2 sub arrays is made so that the sorted sub arrays do not need to be merged later.
- This is accomplished by rearranging the elements in a[1:n] such that a[I]<=a[j] for all I between 1 & n and all j between (m+1) & n for some m, 1<=m<=n.
- Thus the elements in a[1:m] & a[m+1:n] can be independently sorted.
- No merge is needed. This rearranging is referred to as partitioning.
- Quick sort is otherwise called as in place sort or partition exchange sorting

Divide and conquer method:
Given a function to compute on 'n' inputs the divide-and-conquer strategy suggests splitting the inputs into 'k' distinct subsets, 1<k<=n, yielding 'k' sub problems. These sub problems must be solved, and then a method must be found to combine sub solutions into a solution of the whole. If the sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied.

Principle:

1. Choose any number in the array and name it as partition element or pivot element (P). For simplicity we take the first number as the ***pivot element P***.
2. With respect to P, divide the array into two partitions i.e., left, right partition. The numbers which are less than P are placed in the left side of P. And the numbers which are greater than P are placed in the right side of P, for eg., *If the position of P is J*, then it satisfies the following conditions.

   i) Each number in the position from 1 to J-1 are less than or equal to P.

   ii) Each number in the position from J+1 to n are greater than or equal to P.

   iii) The Pivot element is placed in its proper position.
3. We can repeat the steps 1 and 2 for the left and right partition.

Algorithm Quicksort(left,right)

```
//Sort the elements a[left],….a[right] which resides
//is the global array a[1:n] into ascending
//order; a[n+1] is considered to be defined
// and must be >= all the elements in a[1:n]
{
        if(left<right) then // If there are more than one element
        {
            // divide p into 2 subproblems
                j=Partition(a,left,right);
                        //'j' is the position of the partitioning element.
            //solve the subproblems.
```

```
        Quicksort(left,j-1);
        Quicksort(j+1,right);
    //There is no need for combining solution.
    }
}
```

Algorithm Partition(a, left, right)

```
//within a[m],a[m+1],…..,a[left-1] the elements are rearranged
if(left<right)
{
    pivot=a[left];
    i=left;  j=right+1;
        repeat
        {
            repeat
                i=i+1;
            until (a[i]<pivot)&&(i<=right);
            repeat
                j=j-1;
            until (a[j]>pivot)&&(j>=left);
            if(i<j) Interchange(a,i,j);
        }until(i>=j);
        interchange(a,left,j);
        return j;
}
```

Algorithm Interchange(a,i,j)

```
//Exchange a[i] with a[j]
{
p=a[i];
a[i]=a[j];
a[j]=p;
}
```

QUICK SORT TRACE OUT:

12	56	4	89	45	34	21	5	3	71
Pivot = 12	i								j

56<12 => False 71 > 12 => True

12	56	4	89	45	34	21	5	3	71
	i							j	

3 > 12 => False

Since i < j, interchange a[i] and a[j] swap 89 & 5

12 3 4 89 45 34 21 5 56 71

i i i j j

4<12=> T 89<12=>F 5>12 => F

Since i< j, interchange a[i] and a[j]

12 3 4 5 45 34 21 89 56 71

i i j

swap 12 & 5 45<12 => F

12 3 4 5 45 34 21 89 56 71

j i j j j

5>12 => F (45>12=>T 34>12=>T 21>12=>T, so j decrements)

Since i < j becomes false,
Interchange a[j] and pivot

5	3	4	12	45	34	21	89	56	71

Here pivot 12 is placed at the correct position. The elements at left of 12 are lesser and right of 12 are greater.

Take the elements to the left of 12

5 3 4

Pivot =5 i i i

3<5=>T 4<5=> T

5 3 4

j i

4>5 => F

Since i < j becomes false,
Interchange a[j] and pivot,

4 3 5

Here pivot 5 is placed at correct position. There are No Elements to right of 5.
Consider elements at left of 5. (i.e) 4 3 Here **Pivot = 4**

i i

3< 4 => T

4 3

j i

Since i<j false,
Swap a[j] and pivot
3 **4**
Pivot is placed at correct position. Since only one element to left of 4, it is in correct position.
3 **4**
So far, the elements in correct position are
3 **4** **5** **12**

Still the elements to right of 12 are not sorted. So take those elements,

45 34 21 89 56 71

Pivot = 45 i i i

34<45=>T 21<45=>T 89<45=>F

45 34 21 89 56 71

j i j j j

21>45 => F 89>45=>T 56>45=>T 71>45=>T

Since i < j becomes false,
Swap a[j] and pivot
21 34 **45** 89 56 71
The pivot 45 is placed at correct position.

Take elements at left of 45

21 34

Pivot = 21 i

34<21=>F

21 34

j i j

21>21=>F 34>21=>T

Since i<j becomes false
Swap a[j] and pivot. Here no changes

21 34

Since there is only one element to right of pivot 21, it is in correct position.

Hence,

21 **34**

Take elements at right of 45

89 56 71

Pivot =89 i i i

56<89=>T 71<89=>T

89 56 71

j i

71>89=>F
Since i<j becomes false,
Swap a[j] and pivot.
71 56 **89**
89 is placed at correct position.
Now sort the element to left of 89.
71 56

Pivot =71 i i

56<71=>T

71 56

j i

56>71=>F

Since i<j becomes false,
Swap a[j] and pivot
56 71
Since there only one element to right of 56, it is in correct position
56 **71**

Hence all elements are placed at correct position.

3 4 5 12 21 34 45 56 71 89

6. Describe about Stassen's Matrix Multiplication and Stassen's Formula.

Conventional Method:

Let A and B be the 2 n*n Matrices. The product matrix C=AB is calculated by using the formula,
C (i ,j) = A(i,k)* B(k,j) for all 'i' and and j between 1 and n.

The time complexity for the matrix Multiplication is **O(n^3).**

Divide & Conquer Method:

- ✓ Divide and conquer method suggest another way to compute the product of n*n matrix.
- ✓ We assume that n is a power of 2 .In the case n is not a power of 2 ,then enough rows and columns of zero can be added to both A and B .So that the resulting dimension are the powers of two.
- ✓ If n=2 then the following formula as a computed using a matrix multiplication operation for the elements of A & B.
- ✓ If n>2,Then the elements are partitioned into sub matrix n/2*n/2..since 'n' is a power of 2 these product can be recursively computed using the same formula. This Algorithm will continue applying itself to smaller sub matrix until 'n' become suitable small(n=2) so that the product is computed directly .

- To Compute the product C=A*B The formulas are

$$\begin{pmatrix} A11 & A12 \\ A21 & A22 \end{pmatrix} * \begin{pmatrix} B11 & B12 \\ B21 & B22 \end{pmatrix} = \begin{pmatrix} C11 & C12 \\ C21 & C22 \end{pmatrix}$$

C11=A11B11+A12B21

C12=A11B12+A12B22

C21=A21B11+A22B21

C22=A21B12+A22B22

Example: Consider the two 2 X 2 Matrices A & B Find the Product of the Matrix C=A X B

$$\text{LET A=} \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix} \qquad \text{Let B=} \begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix}$$

C11=A11*B11+A12*B21 therefore C11= 2*4 + 2 *4= 16

C12=A11*B12+A12*B22 therefore C12= 2*4 + 2 *4= 16

C21=A21*B11+A22*B21 therefore C13= 2*4 + 2 *4= 16

C22=A21*B12+A22*B22 therefore C14= 2*4 + 2 *4= 16

$$C= \begin{pmatrix} 16 & 16 \\ 16 & 16 \end{pmatrix}$$

To compute AB using the equation we need to perform 8 multiplication of n/2*n/2 matrix and from 4 addition of n/2*n/2 matrix. Since two n/2*n/2 matrices can be added in time cn^2 for some constant c, the overall computing time T(n) of the resulting ivie and conquer algorithm is given by the recurrence

$$T(n)= \begin{cases} b & n<=2 \text{ a \& b are constant} \\ 8T(n/2)+cn^2 & n>2 \end{cases}$$

Drawbacks:

- No better improvement in D & C method when comared to conventional method.
- Matrix multiplication is more expensive then the matrix adddition O(n^3).We can attempt to reformulate the equation for cij so as to have fewer multiplication and possibly more addition .

Volker Strassen's Method:

- Strassen has discovered a way to compute the cij of equation (2) using only 7 multiplication and 18 addition or subtraction.
- Strassen's formulas are

P=(A11+A12)(B11+B22)

Q=(A12+A22)B11

R=A11(B12-B22)

S=A22(B21-B11)

T=(A11+A12)B22

U=(A21-A11)(B11+B12)

V=(A12-A22)(B21+B22)

Cij is computed using the following formulas:

C11=P+S-T+V

C12=R+T

C21=Q+T

C22=P+R-Q+V

Example: Consider the two 2 X 2 Matrices A & B Find the Product of the Matrix C=A X B

$$\text{LET } A= \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix} \qquad \text{Let } B= \begin{pmatrix} 4 & 4 \\ 4 & 4 \end{pmatrix}$$

Substituting A11,A12,A21,A22 = 2 & B11,B12,B21,B22 =4

P=(A11+A12)(B11+B22) => (2+2)*(4+4) => 32

Q=(A21+A22)B11 => (2+2)*4 => 16

R=A11(B12-B22) => 2(4-4) => 0

S=A22(B21-B11) => 2(4-4) => 0

T=(A11+A12)B22 =>(2+2)*4 =>16

U=(A21-A11)(B11+B12) =>(2-2)*(4+4) =>0

V=(A12-A22)(B21+B22) =>(2-2)*(4+4) =0

Cij is computed using the following formulas:

C11=P+S-T+V => 32+0-16+0 => 16

C12=R+T => 0+16 =>16

C21=Q+S => 16+0 =>16

C22=P+R-Q+V => 32+0-16+0=>16

Therefore $C=\begin{pmatrix} 16 & 16 \\ 16 & 16 \end{pmatrix}$

- Computation of P,Q,R,S,T,U,V need 7 multiplications and 10 additions/subtractions.
- Computation of Cij need 8 additions/subtractions.

Time Complexity:

$$T(n)=\begin{cases} b & n\leq 2 \\ 7T(n/2)+an^2 & n>2 \text{ where a \& b are constants.} \end{cases}$$

By substitution method,we get $T(n)=O(n^{\log 27}) \approx O(n^{2.81})$

1. What is greedy method give one Example?

- Greedy method is the most straight forward designed technique.
- As the name suggests, they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.

Definition:

- A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.
- A feasible solution that either maximize or minimize a given objectives function is called an optimal solution.
- An optimal solution should have highest profit.

Control algorithm for Greedy Method:

```
Algorithm Greedy (a,n)
//a[1:n] contain the 'n' inputs
{
Solution =0;//Initialise the solution.
For i=1 to n do
{
x=select(a);
if (feasible(solution,x))then
Solution=union (solution,x);
}
return solution;
}
```

Function select( ):

- The function select an input from a[] and removes it.
- The select input value is assigned to x.

Function feasible( ):

- It is a Boolean value function that determines whether x can be included into the solution vector or not.

Function union( ):

- It combines x with the solution and updates the objective function.

Function greedy( ):

- The function Greedy describes the essential way that a greedy algorithm will look,once a particular problem is chosen and the functions select( ) , feasible( ) & union( ) are properly implemented.

Example

- Suppose we have in a country the following coins are available :
  Dollars(100 cents)

  Quarters(25 cents)

  Dimes(10 cents)

  Nickel(5 Cents)

  Pennies(1 cent)

- Our aim is paying a given amount to a customer using the smallest possible number of coins.
- For example if we must pay 276 cents possible solution then,
  Feasible Solution:
  ❼ 1 dollar +7 Quarters+ 1 pennies➔9 coins

- 2 dollars +3 Quarters +1 Pennies ➔6 coins
- 2 dollars +7 Dimes +1 Nickel +1 Pennies ➔11 coins.

Optimal Solution: 6 coins

2. Explain the Knapsack problem using Greedy Method.

Greedy method:

- ✓ Greedy method is the most straight forward designed technique.
- ✓ As the name suggests, they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.
- ✓ A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.
- ✓ A feasible solution that either maximize or minimize a given objectives function is called an optimal solution.

Problem statement:

We are given n objects and knapsack or bag with capacity M.
Each object i have a weight wi and profit pi, where i varies from 1 to n.

- The problem is we have to fill the bag with the help of n objects and the resulting profit should be maximum.
- Total weight of all chosen objects to be almost M.
- Formally the problem can be stated as
  Maximize $\sum xipi$

  subject to $\sum XiWi<=M$ where $1<=i<=n$

Where xi is the fraction of object and it lies between 0 to 1.

- There are so many ways to solve this problem which will give many feasible solutions for which we have to find the optimal solution.
- But in this algorithm, it will generate only one solution which is going to be feasible as well as optimal.

STEPS:

1. First we find the profit & weight rates of each and every object and sort it according to the descending order of the ratios.
2. Select an object with highest p/w ratio and check whether its weight is lesser than the capacity of the bag.
3. If so place 1 unit of the first object and decrement .the capacity of the bag by the weight of the object you have placed.
4. Repeat the above steps until the capacity of the bag becomes less than the weight of the object you have selected .In this case, place a fraction of the object and come out of the loop whenever you selected.

ALGORITHM:

```
Algorithm Greedy knapsack (m,n)
//P[1:n] and the w[1:n]contain the profits
// & weight respectively of the n objects ordered
//such that p[i]/w[i] >=p[i+1]/W[i+1]
//m is the Knapsack size and x[1:n] is the solution vector.
        {
            for i:=1 to n do a[i]=0.0;
                u:=m;
                 for i: =1 to n do
                  {
                        if (w[i]>u) then break;
                         x[i]=1.0;u=u-w[i];
                  }
     if(i<=n)then x[i]=u/w[i];
        }
```

Method 1: Selecting the fractions Randomly

- Fraction of each item is chosen at random and the total profit is analyzed. The set which yields the maximum profit is taken as the solution.

Method 2:

- Include the item with largest profit.
- In this method fill the knapsack by including next the object with largest profit.
- If an object under consideration doesn't fit, then a fraction of it is included to fill the knapsack.
- Thus each time an object is included into knapsack the largest possible increase in profit value is obtained.

Method 3:

- Include the item with the least weight into the sack.
- Consider the objects in order of non decreasing weights w_i
- Even though capacity is used slowly, profits aren't coming in rapidly enough.

Method 4:

- This method strives to achieve a balance between the rate at which the profit increases and the rate at which capacity is used.
- At each step include the object which has the maximum profit per unit of capacity used. This means that objects are considered in order of the ratio pi/wi
- This method yields an optimal solution.

Example:

Let n=3 and the capacity m=20

P1	**P2**	**P3**
25	24	15
W1	**W2**	**W3**
18	15	10

Method 1: <u>Choose The Ratios Randomly</u> for example X1 =1/2 X2 =1/3 X3=1/4

The problem can be stated as Maximize ∑ xi *pi subject to ∑Xi*Wi<=M where 1<=i<=n
We have to find i=1 to n ∑ Pi Xi =.> P1X1+ P2X2 + P3X3
We have to find i=1 to n ∑ Wi Xi =.> W1X1+ W2X2 + W3X3

P1X1= 25/2 =12.5 P2X2= 24/3 =8 P3X3=15/4= 3.75
W1X1=18/2=9 W2X2=15/3=3 W3X3=10/4=2.5

Therefore W1X1+W2X2+W3X3 = 9+3+2.5
= 16.5≤ 20
Therefore all the three items are added into the sample.
Therefore Total Profit = P1X1+P2X2+P3X3
= 12.5 +8+ 3.75
= 24.25

Method 2: Fill the knapsack with objects selected based on the decreasing order of profit order (25,24,15) i.e(1, 2, 3)

The problem can be stated as Maximize ∑ xi *pi subject to ∑Xi*Wi<=M where 1<=i<=n
We have to find i=1 to n ∑ Pi Xi =.> P1X1+ P2X2 + P3X3
We have to find i=1 to n ∑ Wi Xi =.> W1X1+ W2X2 + W3X3

Adding Item 1: X1=1 W1X1= 18<=20 P1X1=25
Adding item 2:
X2=??
Remaining capacity = (20-18) =2
Therefore X2 = (Remaining capacity/W2) X2= 2/15

W2X2 =15 (2/15)= 2≤ 20

Therefore Total Weight W1X1+W2X2= 18+2
=20≤20
P2X2= 3.2 Therefore Total profit is P1X1 +P2X2 = 28.2

Method 3: Add objects in the order of increasing weights order (10,15,18) i.e(3,2,1)

W3= 10 W2=18 W1=15
P3=15 P2=24 P1=25
Adding item 3:

W3X3 =10 Here X3=1 P3X3= 15

Adding item 2: W2X2+W3X3≤20

W2X2= 20-10
X2= 10/W2 => X2= 10/15 = 2/3

Therefore W2X2 = 15(2/3) = 10 P2X2= (24) 2/3 =16

Therefore Total Weight W3X3+W2X2= 10+10 =20

Therefore Total profit is P3X3+P2X2 = 15+16 =31

Therefore the Total profit =31.

Method 4: Add objects in the decreasing order of ratio Pi/Wi

P1/W1 = 25/18 =1.38
P2/W2 =24/15 =1.6
P3/W3 =15/10 =1.5

Order (1.6,1.5,1.38) i.e order (2,3,1)

Adding item 2:

W2X2 = 15 here X2 =1 P2X2= 24(1) =24

Adding item 3:

W3X3 +W2X2 ≤ 20
W3X3 ≤ 201 -15 Therefore X3= 5/W3 => X3= 1/2

Therefore W3X3 =(10) (0.5) =5

Therefore Total Weight W3X3+W2X2= 5+15 =20

Therefore Total profit is P3X3+P2X2 = 31.5

Method	Order	**(X1,X2,X3)**	**∑ Wi Xi**	**∑ Pi Xi**
1	-------	(1/2,1/3,1/4)	16.5	24.25
2	(1,2,3)	(1,2/15,0)	20	28.2
3	(3,2,1)	(0,2/3,1)	20	31
4	(2,3,1)	(0,1,1/2)	20	31.5

Method 4 gives a maximum profit of 31.5 The items in the sack are 2 and 3

3. Explain about Minimum Spanning Tree.

Greedy method:

- ✓ Greedy method is the most straight forward designed technique.
- ✓ As the name suggests, they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.
- ✓ A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.
- ✓ A feasible solution that either maximize or minimize a given objectives function is called an optimal solution.

Problem statement:

Let G (V, E) be an undirected connected graph with vertices 'v' and edge 'E'.
A sub-graph t= (V, E') of G is a *Spanning tree* of G iff 't' is a tree.
The problem is to generate a graph G= (V, E) where 'E' is the subset of E and G is a Minimum spanning tree. Each and every edge will contain the given non-negative length. Connect all the nodes with edge present in set E' and weight has to be minimum.

Note: We have to visit all the nodes.

The subset tree (i.e.) any connected graph with 'N' vertices must have at least N-1 edges and also it does not form a cycle.

Definition:

- ✓ A spanning tree of a graph is an undirected tree consisting of only those edges that are necessary to connect all the vertices in the original graph.
- ✓ A Spanning tree has a property that for any pair of vertices there exist only one path between them and the insertion of an edge to a spanning tree form a unique cycle.

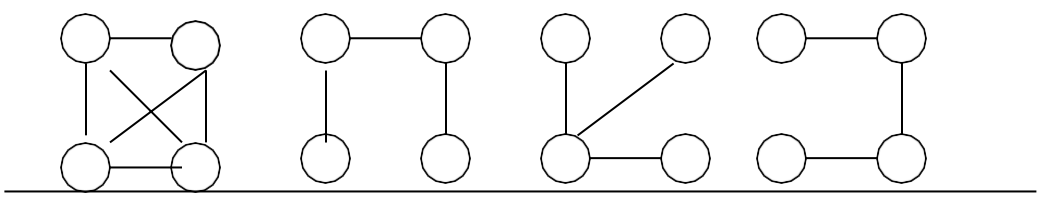


Fig: Graph and its spanning trees,

Application of the spanning tree:

1. Analysis of electrical circuits. 2. Shortest route problems.

Minimum cost spanning tree:

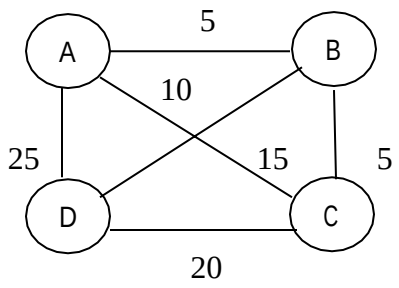
- ✓ The cost of a spanning tree is the sum of cost of the edges in those trees.
- ✓ There are 2 methods to determine a minimum cost spanning tree. They are
  1. Prim's Algorithm. 2. Kruskal's Algorithm

PRIM'S ALGORITHM TO FIND THE MINIMUM SPANNING TREE OF A GRAPH

Algorithm:

1. Construct a cost matrix for a given graph G.
   // Cost matrix contains a cost of each vertex to every other vertex in graph G
2. Start at any arbitrary vertex. Add that vertex to the tree T and mark it as visited
3. The edge (u,v) can be added to the tree T, if
   i) It is an edge with minimum cost
   ii) It should not create a cycle in T
   iii) The newly added vertex doesn't present in T already. If so, that vertex v is marked as visited.

Example:



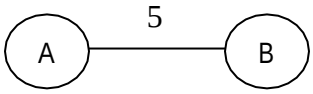
Cost matrix:

	A	B	C	D
A	-	5	10	25
B	5	-	5	15
C	10	5	-	20
D	25	15	20	-

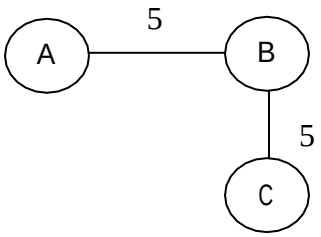
Initially spanning tree T is Null
Consider, the arbitrary vertex chosen is A
Hence A is added to T and it is marked as visited

Step 1: T

Step 2:
The edge A,B is an edge with minimum cost in row A. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.

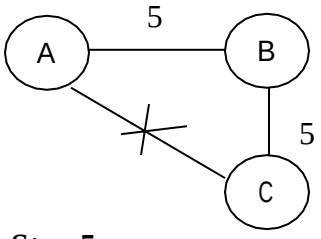


Step 3:
The edge B,C is an edge with next minimum cost in row A and B. Adding of that edge doesn't form any cycle. Hence it is added to the tree T.



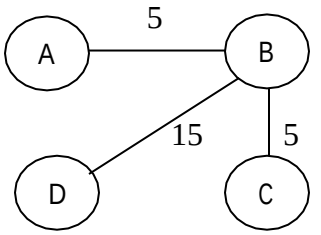
Step 4:

The edge A,C is an edge with next minimum cost in row A, B and C. Adding an edge between A and C creates a loop in spanning tree T. Hence it is ignored.



Step 5:

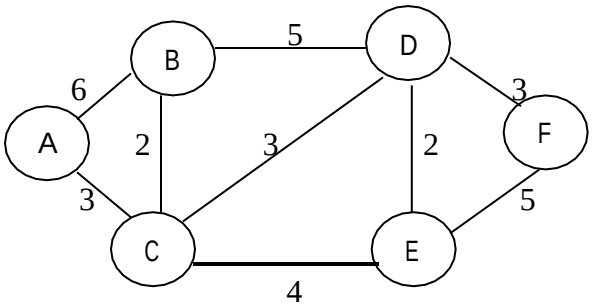
The edge B,D is an edge with next minimum cost in row A, B and C. Adding of that edge doesn't form any cycle. Hence it is added to the tree T.



In minimum spanning tree T, All vertices are visited exactly once.

Minimum cost of visiting all vertices of given graph G exactly once is 25.

Example 2:



Cost matrix:

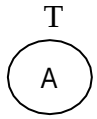
	A	B	C	D	E	F
A	-	6	3	∞	∞	∞
B	6	-	2	5	∞	∞
C	3	2	-	3	4	∞
D	∞	5	3	-	2	3
E	∞	∞	4	2	-	5
F	∞	∞	∞	3	5	-

Initially spanning tree T is Null

Consider, the arbitrary vertex chosen is A

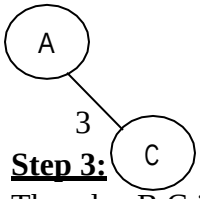
Hence A is added to T and it is marked as visited

Step 1:



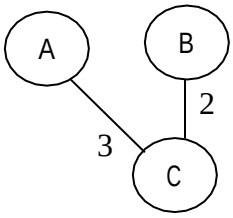
Step 2:

The edge A,C is an edge with minimum cost in row A. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.



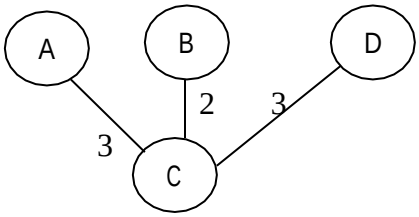
Step 3:

The edge B,C is an edge with next minimum cost in row A and B. Adding of that edge doesn't form any cycle. Hence it is added to the tree T.



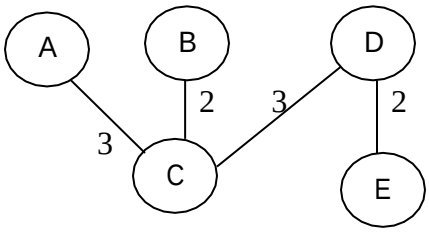
Step 4:

The edge C,D is an edge with next minimum cost in row A, B and C. Adding of that edge doesn't form any cycle. Hence it is added to the tree T.



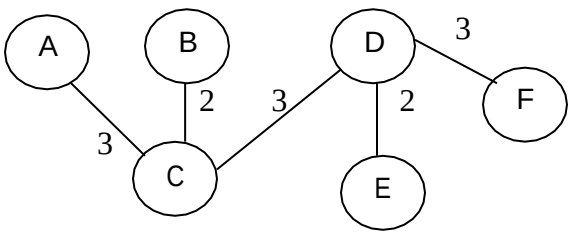
Step 5:

The edge D,E is an edge with next minimum cost in row A, B, C and D. Adding of that edge doesn't form any cycle. Hence it is added to the tree T.



Step 6:

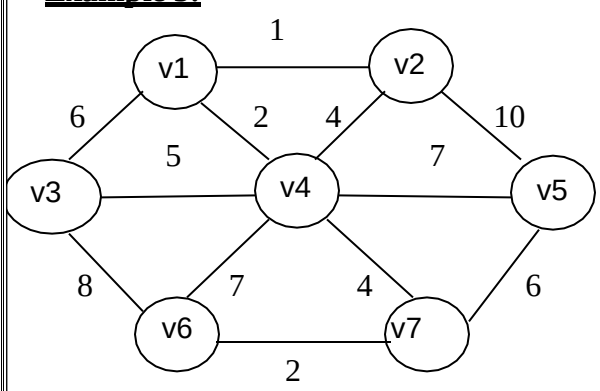
The edge D,F is an edge with next minimum cost in row A, B, C, D and E. Adding of that edge doesn't form any cycle. Hence it is added to the tree T.



In above minimum spanning tree T, All vertices are visited exactly once.

Minimum cost of visiting all vertices of given graph G exactly once is 13.

Example 3:



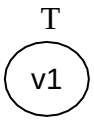
Cost matrix:

	v1	**v2**	**v3**	**v4**	**v5**	**v6**	**v7**
v1	-	1	6	2	∞	∞	∞
v2	1	-	∞	4	10	∞	∞
v3	6	∞	-	5	8	∞	∞
v4	2	4	5	-	7	7	4
v5	∞	10	∞	7	-	∞	6
v6	∞	∞	8	7	∞	-	2
v7	∞	∞	∞	4	6	2	-

Initially spanning tree T is Null

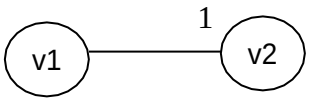
Consider, the arbitrary vertex chosen is v1
Hence v1 is added to T and it is marked as visited

Step 1:



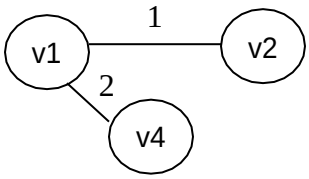
Step 2:

The edge v1,v2 is an edge with minimum cost in row v1. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.



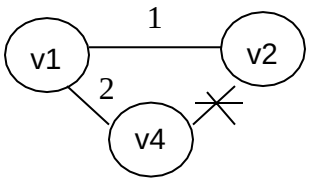
Step 3:

The edge v1,v4 is an edge with minimum cost in row v1 and v2. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.



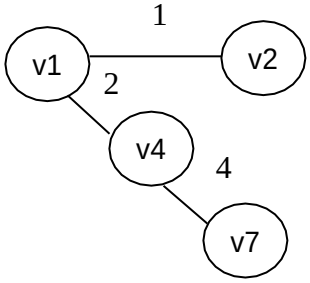
Step 4:

The edge v2,v4 is an edge with minimum cost in row v1, v2 and v4. **Adding edge v2,v4 form a cycle.** Hence this edge is ignored.

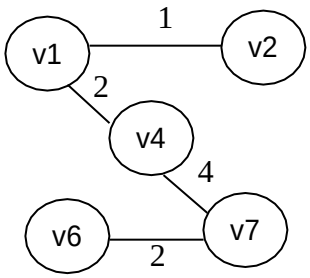


Step 5:

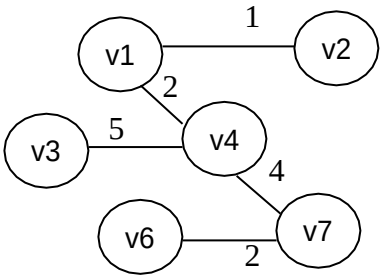
The edge v4,v7 is an edge with minimum cost in row v1, v2 and v4. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.



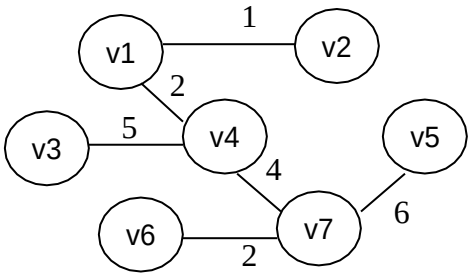
Step 6: The edge v7,v6 is an edge with minimum cost in row v1, v2, v4 and v7. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.



Step 7: The edge v4,v3 is an edge with minimum cost in row v1, v2, v4, v6 and v7. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.



Step 8: The edge v7,v5 is an edge with minimum cost in row v1, v2, v3, v4, v6 and v7. Adding of that edge doesn't form any cycle. Hence it is added to the tree T and it is marked.



In above minimum spanning tree T, All vertices are visited exactly once.

Minimum cost of visiting all vertices of given graph G exactly once is 20.

KRUSKAL ALGORITHM TO FIND MINIMUM SPANNING TREE OF GRAPH

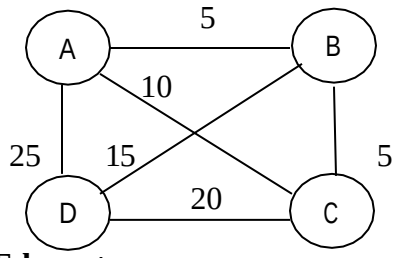
Greedy method:

- ✓ Greedy method is the most straight forward designed technique.
- ✓ As the name suggests, they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.
- ✓ A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.
- ✓ A feasible solution that either maximize or minimize a given objectives function is called an optimal solution.

Algorithm:

T = Null. // T is a minimum spanning tree T
E is an edge set which contains all edges in G from lowest cost to highest cost
While((Number of edges in T < n – 1) && (E ! = empty))
    Choose an edge (u, v) from E with lowest cost
    Delete (u,v) from E
    If (u, v) doesn't creates a cycle in T
        Add that edge edge (u,v) to T
    Else
        Discard that edge (u,v)
    End if
End while

Example:

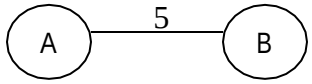


Edge set:

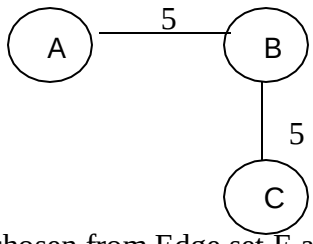
Edges	AB	BC	AC	BD	DC	AD
Cost	5	5	10	15	20	25

Steps:

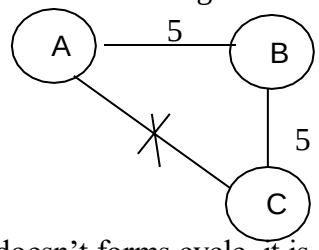
Initially T is null. Number of vertices n = 4. Hence n-1 = 3 Since, Number of edges in T is 0 < 3 and E ! = empty Edge AB is chosen from Edge set E and it is deleted. Since adding of edge AB doesn't form cycle, it is added to T



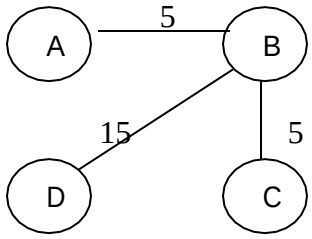
Now, Number of edges in T is 1 < 3 and E ! = empty. Edge BC is chosen from Edge set E and it is deleted. Since adding of edge BC doesn't form cycle, it is added to T



Now, Number of edges in T is 2 < 3 and E ! = empty. Edge AC is chosen from Edge set E and it is deleted. Since adding of edge AC forms cycle, it is ignored



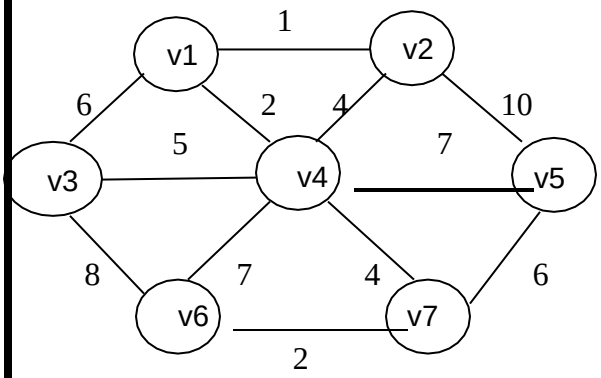
Edge BD is chosen from Edge set E and it is deleted. Since adding of edge BD doesn't forms cycle, it is added to T



In above minimum spanning tree T, All vertices are visited exactly once.

Minimum cost of visiting all vertices of given graph G exactly once is 25.

<u>Example 2:</u>

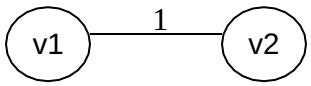


<u>Edge set:</u>

Edges	**v1,v2**	**v1,v4**	**v6,v7**	**v2,v4**	**v4,v7**	**v3,v4**	**v1,v3**	**v5,v7**	**v4,v5**	**v4,v6**	**v3,v6**	**v2,v5**
Cost	1	2	2	4	4	5	6	6	7	7	8	10

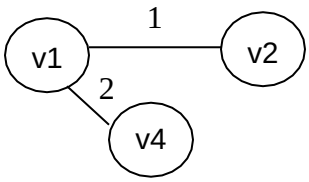
<u>Step 1:</u>

Initially T is null. Number of vertices n =7. Hence n-1 = 6 Edge v1,v2 is chosen from Edge set E and it is deleted. Since adding of edge v1v2 doesn't form cycle, it is added to T



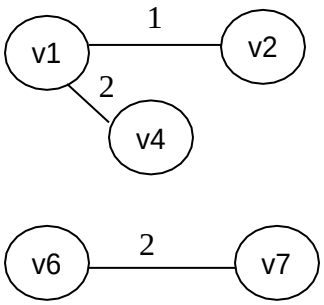
Step 2:

Edge v1,v4 is chosen from Edge set E and it is deleted. Since adding of edge v1v4 doesn't form cycle, it is added to T



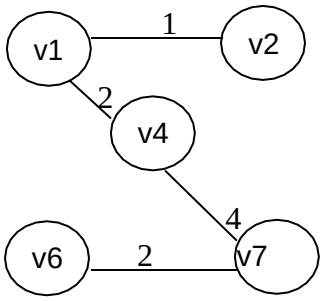
Step 3:

The edge v6,v7 is chosen from Edge set E and it is deleted. Since adding of edge v6,v7 doesn't form cycle, it is added to T

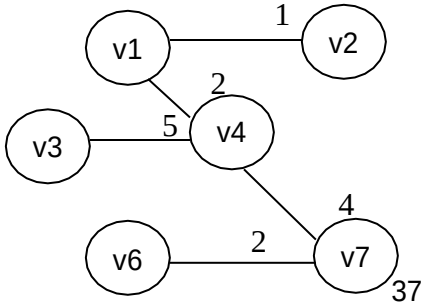


Step 4: The edge v2,v4 is chosen from Edge set E and it is deleted. Since adding of edge v2,v4 form cycle, it is ignored

Step 5: The edge v4,v7 is chosen from Edge set E and it is deleted. Since adding of edge v4,v7 doesn't form cycle, it is added to T

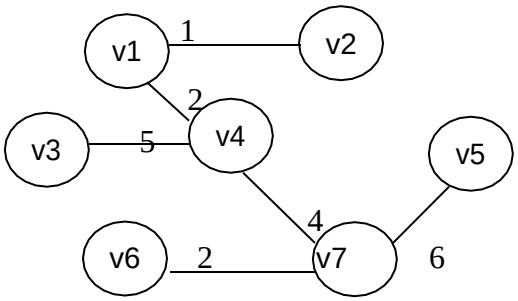


Step 6: The edge v3,v4 is chosen from Edge set E and it is deleted. Since adding of edge v3,v4 doesn't form cycle, it is added to T



Step 7:
The edge v1,v3 is chosen from Edge set E and it is deleted. Since adding of edge v1,v3 form cycle, it is ignored

Step 8:
The edge v5,v7 is chosen from Edge set E and it is deleted. Since adding of edge v5,v7 doesn't form cycle, it is added to T



In above minimum spanning tree T, All vertices are visited exactly once.

Minimum cost of visiting all vertices of given graph G exactly once is 20.

4. Explain the Single source shortest path algorithm with Example.

Greedy method:

- ✓ Greedy method is the most straight forward designed technique.
- ✓ As the name suggests, they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.
- ✓ A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.
- ✓ A feasible solution that either maximize or minimize a given objectives function is called an optimal solution.

Problem Statement:

This problem is a common path finding problem on directed graph. We are given a directed graph G(V,E)in which each arc has a non-negative label and one vertex is specified as the source.

Our problem is to determine the cost of the shortest path from the source to every other vertex in V where the length of the path is the sum of the cost of the arcs on the path. This problem is often called as the single source shortest path problem.

To solve this problem we shall use a greedy technique often known as Dijkstra algorithm. This algorithm works by maintaining a set S of vertices whose shortest distance from the source is already known. Initially S contains only the source vertex. At each step we add to So remaining vertex V whose distance from the source is as short as possible.

Assuming all arcs have non-negative cost, we can always find a shortest path from the source to V that passes only through vertices in S, call such a path special. At each step of an algorithm we us

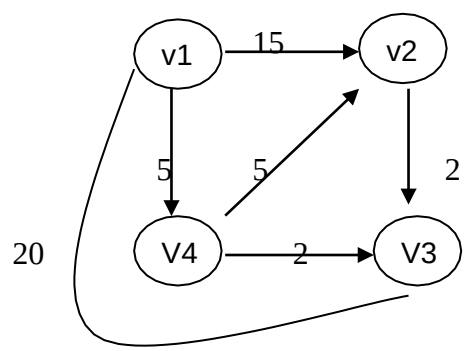
an array 'D' to record the length of the shortest special path to each vertex. Once S includes all vertices, all paths are special. So D hold the shortest distance from the source to each vertex.

```
Algorithm Shortest path (v, cost, dist, n)

//dist [j], 1 ≤ j≤ n is a set of length of shortest.

{
        for i=1 to n do
        {
            s[i]=0; dist [i]=cost[v][i];
        }
        s[v]=1; dist[v]=0;
            for(num=2; num≤ n;num++)
            {
              min=9999;
              for(j=1;j≤n; j++)
                  {
                     if((s[j]==0) &&(min>dist[j]))
                     {
                       min=dist[j];

                       u=j;
                     }
                  }
            }
    s[u]=1;
  for (w=1;w≤n; w++)
      {
         if((s[w]==0) && (dist[w]>dist[u]+cost[u][w]))
          {
            dist[w]=dist[u]+cost[u][w];
          }
}
```



Example: Find the single source shortest path for the given directed graph.

Solution:

S.no	Edge	Feasible Path	Cost Obtained
1	V1-V1	0	0
2	V1-V2	V1-V2	15

		V1-V4-V2	10
3	V1-V3	V1-V3	20
		V1-V2-V3	7
		V1-V4-V2-V3	12
4	V1-V4	V1-V4	20

OPTIMAL SOLUTION:

S.NO	EDGE	MINIMUM COST
1	V1-V1	0
2	V1-V2	10
3	V1-V3	7
4	V1-V4	20

5. Explain about the optimal storage on tapes using greedy method.

Greedy method:

- ✓ Greedy method is the most straight forward designed technique.
- ✓ As the name suggests, they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.
- ✓ A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.
- ✓ A feasible solution that either maximize or minimize a given objectives function is called an optimal solution.

Problem statement:

- ✓ We have n no of programs. The length of the tape l & length of each program li, $1 \le i \le n$.
- ✓ The objective of the problem is to store 'n' no of programs on a tape with the constraint : Sum of the length of all the programs should not exceed the length of the tape l.i.e)All the programs can be stored on the tape iff the sum of the length of the program is atmost l.
- ✓ Assume whenever a program is retrieved from the tape, tape is initially positioned at the front.
- ✓ Time needed (tj)to retreive the program lj is proportional to $\sum$ li*k where k=1 to j.

- ✓ Program P1 having length l1 takes t1 time to store it.
- ✓ Program P2 having length l2 takes t2 time to store it.
- ✓ Program P3 having length l3 takes t3 time to store it.
- ✓ We have to store l1,l2,l3 in l;
- ✓ If all the programs are equally retrieved,then

Mean retreival time d = (1/n) ∑ tj ,1≤j≤n

d = (t1+t2+.... +tn)/n

- Our aim is to minimize the mean retreival time. To minimize MRT we have to find the permutation for n programs.i.e)We must consider the order in which programs are saved.

Example:

P1---l_1---> 5 P2---l_2--->10 P3---l_3---> 3

Possible orderings=n!=3!=6

I	d(i)	MRT
1 , 2 , 3	5+15+!8	38
1 , 3 , 2	5+8+18	31
2 , 3 , 1	10+13+18	41
2 , 1 , 3	10+15+18	43
3 , 1 , 2	3+8+18	29(min)
3 , 2 ,1	3+13+18	34

The 5th order 3,2,1 takes minimum MRT 29.So that if the programs are stored in the sequence 3,2,1 then it is an optimal solution.

Analysis:

The greedy method simply requires to store the programs in non decreasing order of their lengths. This ordering can be carried out in O(n log n) time using an efficient sorting algorithm(heap sort).

6. Explain about Optimal Merge pattern with Example.

<u>Greedy method:</u>

- ✓ Greedy method is the most straight forward designed technique.
- ✓ As the name suggests, they are short sighted in their approach taking decision on the basis of the information immediately at the hand without worrying about the effect these decision may have in the future.
- ✓ A problem with N inputs will have some constraints. Any subsets that satisfy these constraints are called a feasible solution.

- ✓ A feasible solution that either maximize or minimize a given objectives function is called an optimal solution.

Problem statement:

- We have 2 sorted files containing n and m records. The file f1 contains n records. The file f2 contains m records.
- These 2 files could be merged together and resultant file is in also sorted order.
- Time taken for merging these two files is O(m+n).
- If we have more than 2 files,then we can merge the files in some order.
- Let us take 4 files say x1,x2,x3&x4. We can merge these 4 files in any order.
- For example:

  1.Merge x1 & x2 => y1

  Merge y1 & x3 => y2

  Merge y2 & x4 => y3

  2.Merge x1 & x2 => y1

  Merge x3 & x4 => y2

  Merge y1 & y2 => y3

- Given n sorted files. We have to sort the n sorted files & store it in a single file in sorted order.

- We can merge the files in any order, but it should take minimum amount of computing time.

Example 1:

n=3

File x1 length=30

File x2 length=20

File x3 length=10

Way 1:

Merge x1 & x2 - 50 record moves

Merge(x1 & x2)&x3 - 60 record moves

Total no of record moves -110 record moves

Way 2:

Merge x2 & x3 - 30 record moves

Merge(x2 & x3)&x1 - 60 record moves

Total no of record moves - 90 record moves

Minimum record moves is in way2(90).So the second merge pattern is faster than first.

By greedy method, optimal merge pattern suggests: At each step merge the 2 smallest files together.

Example 2 :

x1	x2	x3	x4	x5
20	30	10	5	30

Merge x3+x4 → 10+5=15=>z1

Merge z1+x1 → 15+20=35=>z2

Merge x2+x5 → 30+30=60=>z3

Merge z2+z3 → 35+60=95

Total no. of record moves = 205moves

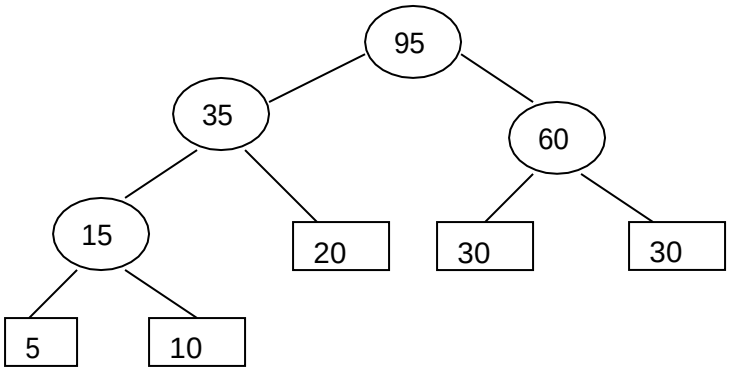
Two Way Merge Pattern:

```
treenode=record{
    treenode*lchild; treenode *rchild;
integer weight;
};
```

```
Algorithm: Tree(n)
//list is a global List of n single node
//binary trees as described above
{
for i=1 to n-1 do
{
pt=new treenode; //Get a new tree node
(pt → lchild):=Least(list); //Merge two trees with smallest length
(pt → rchild):=Least(list);
(pt → weight):= ((pt → lchild) → weight ) + ( pt → lchild) → weight));
Insert (list,pt);
```

```
}
return Least(list);
}
```

1. It is represented by binary merge trees.
2. Leaf nodes are drawn as squares.
3. Nodes are called as external nodes.
4. Remaining nodes are drawn as circles.(ie) internal nodes
5. Each internal node has exactly 2 children. Internal node represents the file obtained by its 2 children.



In the above tree ,square represents a single sorted file and circle represents the merging of 2 files .

UNIT-2 : TWO MARKS

1. **Define the divide an conquer method. [Nov. 2012]**

Given a function to compute on 'n' inputs the divide-and-comquer strategy suggests splitting the inputs in to'k' distinct susbsets, 1<k <n, yielding 'k' subproblems. The subproblems must be solved, and then a method must be found to combine subsolutions into a solution of the whole. If the subproblems are still relatively large, then the divide-and conquer strategy can possibly be reapplied.

2. **Define control abstraction.**

A control abstraction we mean a procedure whose flow of control is clear but whose primary operations are by other procedures whose precise meanings are left undefined.

3. **Write the Control abstraction for Divide-and conquer.**

```
Algorithm DAnd(〉)
{
if small(p) then return S(〉);
else
{
        divide P into smaller instance ρ1, ρ2, ρk, k ε 1;
        Apply D and C to each of these subproblems
        Return combine (DAnd C(ρ1) DAnd C(ρ2),----, DAnd ((ρk));
}
}
```

4. **What is the substitution method?**

One of the methods for solving any such recurrence relation is called the substitution method.

5. **What is the binary search? [Apr 2015]**

If 'q' is always chosen such that 'aq' is the middle element(that is, q=[(n+1)/2), then the resulting search algorithm is known as binary search.

6. **Give computing time for Bianry search? [April. 2011]**

In conclusion we are now able completely describe the computing time of binary search by giving formulas that describe the best, average and worst cases.

Successful searches
⎩(1) ⎩(logn) ⎩(Logn)
best average worst
unsuccessful searches
⎩(logn)
best, average, worst

7. **Define external path length?**

The external path length E, is defines analogously as sum of the distance of all external nodes from the root.

8. **Define internal path length.**

The internal path length 'I' is the sum of the distances of all internal nodes from the root.

9. **What is the maximum and minimum problem?**

The problem is to find the maximum and minimum items in a set of 'n' elements. Though this problem may look so simple as to be contrived, it allows us to demonstrate divide- and-conquer in simple setting.

10. What is the Quick sort?

In quicksort, the division into subarrays is made so that the sorted subarrays do not need to be merged later.

11. Write the Anlysis for the Quick sot.

In analyzing QUICKSORT, we can only make the number of element comparisions c(n). It is easy to see that the frequency count of other operations is of the same order as C(n).

12. Is insertion sort better than the merge sort?

Insertion sort works exceedingly fast on arrays of less then 16 elements, though for large 'n' its computing time is $O(n^2)$.

13. Write a algorithm for straightforward maximum and minimum>

```
algorithm straight MaxMin(a,n,max,min)
//set max to the maximum and min to the minimum of a[1:n]
{
        max := min: = a[i];
        for i = 2 to n do
        {
                if(a[i] >max) then max: = a[i];
                if(a[i] >min) then min: = a[i];

        }
}
```

14. Give the recurrence relation of divide-and-conquer?

The recurrence relation is

$$T(n) = g(n)$$
$$T(n_1) + T(n_2) + ----+ T(n_k) + f(n)$$

15. Write the algorithm for Iterative binary search?

```
Algorithm BinSearch(a,n,x)
//Given an array a[1:n] of elements in nondecreasing
// order, n>0, determine whether x is present
{
        low : = 1;
        high : = n;
        while (low <  high) do
        {
                mid : = [(low+high)/2];
                if(x < a[mid]) then high:= mid-1;
                else if (x >a[mid]) then low:=mid + 1;
                else return mid;
        }
        return 0;
}
```

16. What are internal nodes?

The circular node is called the internal nodes.

17. Describe the recurrence relation of merge sort?

If the time for the merging operation is proportional to n, then the computing time of merge sort is described by the recurrence relation

$$T(n) = \begin{cases} a & n = 1, \text{ a a constant} \\ 2T(n/2) + n & n > 1, \text{ c a constant} \end{cases}$$

18.What is meant by feasible solution? [Nov 2014]

Given n inputs and we are required to form a subset such that it satisfies some given constraints then such a subset is called feasible solution.

19. Write any two characteristics of Greedy Algorithm? [Nov. 2011]

* To solve a problem in an optimal way construct the solution from given set of candidates.
* As the algorithm proceeds, two other sets get accumulated among this one set contains the candidates that have been already considered and chosen while the other set contains the candidates that have been considered but rejected.

20. Define optimal solution?

A feasible solution either maximizes or minimizes the given objective function is called as optimal solution

21. What is Knapsack problem? [April 2011]

A bag or sack is given capacity n and n objects are given. Each object has weight w_i and profit p_i .Fraction of object is considered as x_i (i.e) $0<=x_i<=1$ .If fraction is 1 then entire object is put into sack. When we place this fraction into the sack we get w_ix_i and p_ix_i.

22. Define weighted tree.

A directed binary tree for which each edge is labeled with a real number (weight) is called as weighted tree.

23. What is the use of TVSP?

In places where the loss exceeds the tolerance level boosters have to the placed. Given a network and loss tolerance level the tree vertex splitting problems is to determine an optimal placement of boosters.

24. What is the Greedy choice property?

* The first component is greedy choice property (i.e.) a globally optimal solution can arrive at by making a locally optimal choice.
* The choice made by greedy algorithm depends on choices made so far but it cannot depend on any future choices or on solution to the sub problem.
* It progresses in top down fashion.

25. What is greedy method? [April 2014]

Greedy method is the most important design technique, which makes a choice that looks best at that moment. A given 'n' inputs are required us to obtain a subset that satisfies some constraints that is the feasible solution. A greedy method suggests that one can device an algorithm that works in stages considering one input at a time.

26. What are the steps required to develop a greedy algorithm? [Nov. 2011]

* Determine the optimal substructure of the problem.
* Develop a recursive solution.
* Prove that at any stage of recursion one of the optimal choices is greedy choice.

Thus it is always safe to make greedy choice.

* Show that all but one of the sub problems induced by having made the greedy choice are empty.
* Develop a recursive algorithm and convert into iterative algorithm.

27. What is activity selection problem?

The 'n' task will be given with starting time s_i and finishing time f_i. Feasible Solution is that the task should not overlap and optimal solution is that the task should be completed in minimum number of machine set.

28. Write the specification of TVSP

Let T= (V, E, W) be a weighted directed binary tree where
V =vertex set
E =edge set
W =weight function for the edge.
W =is more commonly denoted as w (i,j) which is the weight of the edge <i,j> ε E.

29. Define forest.

Collection of sub trees that are obtained when root node is eliminated is known as forest

30.What does Job sequencing with deadlines mean?

* Given a set of 'n' jobs each job 'i' has a deadline d_i such that d_i>=0 and a profit p_i such that p_i>=0.
* For job 'i' profit p_i is earned iff it is completed within deadline.
* Processing the job on the machine is for 1unit of time. Only one machine is available.

31. Define post order traversal.

The order in which the TVSP visits the nodes of the tree is called the post order traversal.

32. Write the formula to calculate delay and also write the condition in which the node gets splitted?

To calculate delay

$$d(u)=\max_{v \in c(u)}\{d(v)+w(u, v)\}$$

The condition in which the node gets splitted are:
d(u)+w(u ,v)>δ and also d(u) is set to zero.

33. What is meant by tolerance level?

The network can tolerate the losses only up to a certain limit tolerance limit.

34. When is a task said to be late in a schedule?

A task is said to be late in any schedule if it finishes after its deadline else a task is early in a schedule.

35. Define feasible solution for TVSP.

Given a weighted tree T(V,E,W) and a tolerance limit δ any subset X of V is a feasible solution if d(T/X)<= δ.

36. Define optimal solution for TVSP.

An optimal solution is one in which the number of nodes in X is minimized.

37. Write the general algorithm for Greedy method control abstraction. [Nov. 2011]

```
Algorithm Greedy (a, n)
{
        solution=0;
        for i=1 to n do
        {
                x= select(a);
                if feasible(solution ,x) then
                solution=Union(solution ,x);
        }
        return solution;
}
```

38.Define optimal solution for Job sequencing with deadlines.

Feasible solution with maximum profit is optimal solution for Job sequencing with deadlines.